

AfriChart EMR — Phase 0 Technical Build Guide

Developer Reference Document

Project: AfriChart Technologies EMR — Phase 0 Test Sprint **Developer:** Michael Aleruchi Mgbah / Lymora Tech **Client:** AfriChart Technologies **Budget:** ₦30,000 **Timeline:** 7 calendar days from signing **Date:** 30th May 2026

Table of Contents

1. [What You're Building & Why It Matters](#)
2. [Tech Stack & Justification](#)
3. [Project Setup](#)
4. [Folder Structure](#)
5. [Database Schema](#)
6. [Authentication & Role System](#)
7. [Routing Architecture](#)
8. [Controllers, Services & Repositories](#)
9. [Blade Views & UI Architecture](#)
10. [Design Direction](#)
11. [Validation Rules](#)
12. [Seeders & Demo Data](#)
13. [Search & Filter Implementation](#)
14. [Security Considerations](#)
15. [Testing Checklist](#)
16. [Deployment & Demo Environment](#)
17. [Day-by-Day Build Plan](#)
18. [Deliverables Checklist](#)

1. What You're Building & Why It Matters

Phase 0 is a paid audition. The client is using this sprint to evaluate three things about you:

1. **Code quality** — Is your Laravel code well-structured, clean, and maintainable?
2. **UI design sense** — Can you build something a non-technical receptionist would find intuitive?
3. **Reliability** — Can you deliver on time, on spec?

If he approves Phase 0, you move into Phase 1 (₦500,000). If he doesn't, you walk away with ₦30,000 and no further obligation. Everything you build here should quietly signal: "this developer can handle the full product."

What You're Delivering

#	Deliverable	What It Proves
1	Patient registration form	You can build clean forms with proper validation
2	Patient list page	You can display data cleanly with search
3	Login system with Admin + Doctor roles	You understand auth and RBAC
4	Two separate dashboards	You can build role-based UI
5	Desktop-optimised layout	You can design for the target environment
6	GitHub repository	You write clean, documented, organised code

What You're NOT Building

Do not over-engineer. This is a 7-day sprint. No:

- REST API endpoints (that's Phase 1 — your existing ApiResponse trait stays for later)
- Mobile responsiveness
- Prescription, billing, or consultation modules
- Complex reporting
- Multi-tenancy

Build exactly what's in scope. Build it well.

2. Tech Stack & Justification

Layer	Choice	Why
Framework	Laravel 11 (PHP 8.3)	Already installed. Your primary stack

Layer	Choice	Why
Frontend	Blade + Tailwind CSS 3.x	Server-rendered, fast to build. No SPA complexity for an MVP
Auth scaffold	Laravel Breeze	Gives you login/registration views out of the box. Customise from there
Database	MySQL 8.0	Switch from SQLite (current) to MySQL for production readiness
Icons	Phosphor Icons (via Blade Phosphor Icons package)	Clean, flexible icon set with multiple weights (thin, light, regular, bold, fill, duotone)
Build tool	Vite (already configured)	Default bundler. Handles Tailwind compilation
Architecture	Repository + Service pattern	Already scaffolded in your project. Keep it consistent

Why Not Livewire / Inertia / Vue?

For Phase 0, vanilla Blade + Tailwind is the right call. The scope is small (4-6 pages), the client wants to see code clarity, and adding a JS framework creates complexity with no payoff at this scale. Phase 1 may warrant Livewire for interactive components (live search, modals) but that decision comes later.

Your Existing Architecture Patterns

Your project already has a Repository + Service layer pattern:

```
Controller → Service → Repository → Model
```

- **Controllers** handle HTTP (receive request, return response)
- **Services** handle business logic (generate patient IDs, compute stats)
- **Repositories** handle data access (queries, filters, pagination)
- **Models** handle Eloquent relationships and casts

This is cleaner than fat controllers and shows the client you build for maintainability. Use it consistently throughout Phase 0.

3. Project Setup

Current State

Your `africhart-emr` project is already a Laravel installation with:

- Base controllers, models, requests, repositories, services, and traits scaffolded
- Standard Laravel config files
- Default migrations (users, cache, jobs)
- Git initialised
- Vite configured
- SQLite as the current database

You don't need to start from scratch. You just need to add the Phase 0 dependencies on top.

Step 1: Install Dependencies

```
cd ~/Documents/wayne/web_dev/africhart-emr

# Install Breeze (Blade + Tailwind stack)
composer require laravel/breeze --dev
php artisan breeze:install blade

# Install Phosphor Icons for Blade
composer require codeat3/blade-phosphor-icons

# Install NPM dependencies and build
npm install
npm run build
```

Step 2: Switch from SQLite to MySQL

Your project currently uses SQLite (`database/database.sqlite`). Switch to MySQL for production readiness.

Open `.env` and update:

```
APP_NAME="AfriChart EMR"
APP_ENV=local
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=africhart_emr
DB_USERNAME=root
DB_PASSWORD=your_password
```

Create the database:

```
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS africhart_emr;"
```

You can delete `database/database.sqlite` once MySQL is confirmed working.

Step 3: Run Migrations

```
php artisan migrate
```

Step 4: Verify Everything Works

```
php artisan serve
# Visit http://localhost:8000 – you should see the Laravel welcome page
# Visit http://localhost:8000/login – you should see the Breeze login page
```

Step 5: Commit

```
git add .
git commit -m "feat: add breeze auth, tailwind, phosphor icons, switch to mysql"
```

That's it for setup. Your foundation is already solid — now you build on top of it.

4. Folder Structure

Here's what the project should look like after Phase 0 is complete. Files marked with (new) are what you create. Files marked with (modify) already exist and need changes. Everything else is already there.

```
africhart-emr/
|
├── app/
|   ├── Enums/
|   |   ├── UserRole.php           (new) Enum: admin, doctor
|   |   └── BloodGroup.php        (new) Enum: A+, A-, B+, B-, etc.
|   |
|   ├── Http/
|   |   ├── Controllers/
|   |   |   ├── Auth/             (breeze) Auto-generated by Breeze
|   |   |   └── BaseController.php (existing) Your base controller
```

└─ Controller.php	(existing) Laravel default
└─ DashboardController.php	(new) Role-based dashboard routing
└─ PatientController.php	(new) Patient CRUD
└─ Middleware/	
└─ └─ RoleMiddleware.php	(new) Restricts routes by role
└─ Requests/	
└─ └─ BaseRequest.php	(existing) Your base form request
└─ └─ StorePatientRequest.php	(new) Validation for patient creation
└─ └─ UpdatePatientRequest.php	(new) Validation for patient editing
└─ Models/	
└─ └─ User.php	(modify) Add role field + helpers
└─ └─ Patient.php	(new) Patient model
└─ Repositories/	
└─ └─ BaseRepository.php	(existing) Your base repository
└─ └─ PatientRepository.php	(new) Patient data access
└─ Services/	
└─ └─ BaseService.php	(existing) Your base service
└─ └─ DashboardService.php	(new) Dashboard stats logic
└─ └─ PatientService.php	(new) Patient business logic
└─ Traits/	
└─ └─ ApiResponse.php	(existing) Keep for Phase 1
└─ Providers/	
└─ └─ AppServiceProvider.php	(existing)
└─ database/	
└─ └─ migrations/	
└─ └─ └─ 0001_01_01_000000_create_users_table.php	(existing)
└─ └─ └─ 0001_01_01_000001_create_cache_table.php	(existing)
└─ └─ └─ 0001_01_01_000002_create_jobs_table.php	(existing)
└─ └─ └─ xxxx_add_role_to_users_table.php	(new)
└─ └─ └─ xxxx_create_patients_table.php	(new)
└─ └─ seeders/	
└─ └─ └─ DatabaseSeeder.php	(modify) Call new seeders
└─ └─ └─ UserSeeder.php	(new)
└─ └─ └─ PatientSeeder.php	(new)
└─ resources/	
└─ └─ views/	
└─ └─ └─ layouts/	
└─ └─ └─ └─ app.blade.php	(new) Master layout
└─ └─ └─ components/	
└─ └─ └─ └─ sidebar.blade.php	(new) Sidebar navigation
└─ └─ └─ └─ topbar.blade.php	(new) Top bar

└─ stat-card.blade.php	(new) Dashboard stat card
└─ patient-table.blade.php	(new) Reusable patient table
└─ auth/	
└─ login.blade.php	(modify) Customise Breeze login with A
└─ dashboard/	
└─ admin.blade.php	(new) Admin dashboard
└─ doctor.blade.php	(new) Doctor dashboard
└─ patients/	
└─ index.blade.php	(new) Patient listing with search
└─ create.blade.php	(new) Registration form
└─ edit.blade.php	(new) Edit form
└─ show.blade.php	(new) Patient detail view
└─ routes/	
└─ web.php	(modify) Add role-gated routes
└─ public/	
└─ images/	
└─ africhart-logo.svg	(new) Logo placeholder
└─ README.md	(modify) Setup instructions + demo cre

File Count Summary

Category	New	Modify	Existing (untouched)
Enums	2	0	0
Controllers	2	0	2
Middleware	1	0	0
Form Requests	2	0	1
Models	1	1	0
Repositories	1	0	1
Services	2	0	1
Migrations	2	0	3
Seeders	2	1	0
Blade Views	11	1	0
Routes	0	1	0

Category	New	Modify	Existing (untouched)
Other	2	0	—
Total	28	4	8

5. Database Schema

Users Table (modify existing)

Add a `role` column to the existing Laravel users migration. Create a new migration:

```
php artisan make:migration add_role_to_users_table --table=users
```

```
// Migration: add_role_to_users_table
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('role')->default('doctor')->after('email');
    });
}
```

Patients Table

```
php artisan make:migration create_patients_table
```

```
// Migration: create_patients_table
public function up(): void
{
    Schema::create('patients', function (Blueprint $table) {
        $table->id();

        // Core identity
        $table->string('full_name');
        $table->date('date_of_birth');
        $table->string('phone', 20);

        // Medical basics
        $table->string('blood_group', 5); // A+, A-, B+, B-, AB+, AB-, O+, O-
        $table->text('allergies')->nullable(); // Free text, nullable

        // System fields
    });
}
```



```

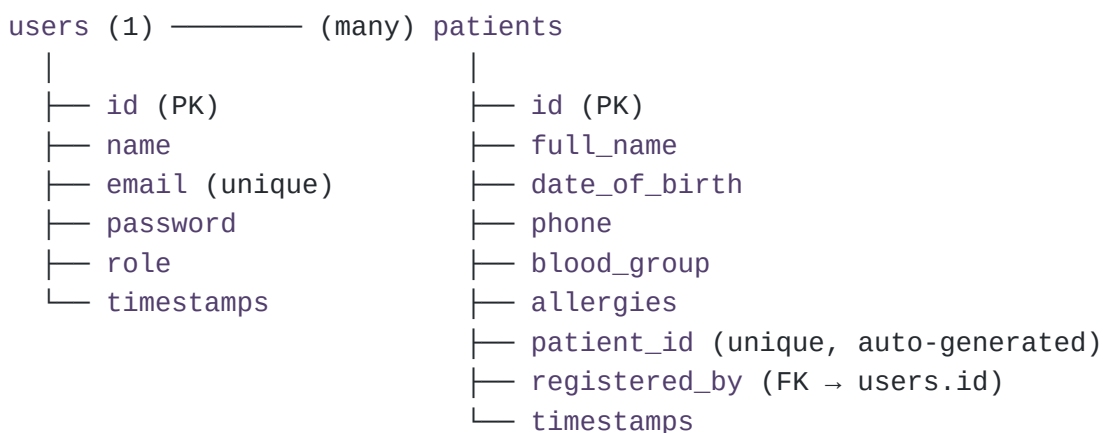
    $table->string('patient_id', 20)->unique(); // Auto-generated unique ID
    $table->foreignId('registered_by')         // Who registered this patient
        ->constrained('users')
        ->onDelete('restrict');                // Never delete a user who reg

    $table->timestamps();

    // Indexes for search performance
    $table->index('full_name');
    $table->index('phone');
    $table->index('patient_id');
  });
}

```

Entity Relationship



Patient ID Format

Generate a human-readable unique ID for each patient. Format: ACH-YYYYMMDD-XXXX

- ACH — AfriChart prefix
- YYYYMMDD — Date of registration
- XXXX — Zero-padded sequential number for that day

Example: ACH-20260530-0001 , ACH-20260530-0002

This goes in the PatientService (see Section 8).

6. Authentication & Role System

Enums

Create two PHP 8.1 enums for type safety:

app/Enums/UserRole.php

```
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case Doctor = 'doctor';

    public function label(): string
    {
        return match($this) {
            self::Admin => 'Admin',
            self::Doctor => 'Doctor',
        };
    }
}
```

app/Enums/BloodGroup.php

```
<?php

namespace App\Enums;

enum BloodGroup: string
{
    case APositive = 'A+';
    case ANegative = 'A-';
    case BPositive = 'B+';
    case BNegative = 'B-';
    case ABPositive = 'AB+';
    case ABNegative = 'AB-';
    case OPositive = 'O+';
    case ONegative = 'O-';

    public function label(): string
    {
        return $this->value;
    }
}
```

User Model Modifications

app/Models/User.php — Add role casting and helper methods:

```
<?php

namespace App\Models;

use App\Enums\UserRole;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
        'role',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    // — Role Helpers —

    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    public function isDoctor(): bool
    {
        return $this->role === UserRole::Doctor;
    }

    // — Relationships —

    public function registeredPatients()
    {
        return $this->hasMany(Patient::class, 'registered_by');
    }
}
```

```
}
```

Role Middleware

app/Http/Middleware/RoleMiddleware.php

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class RoleMiddleware
{
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        if (!auth()->check()) {
            return redirect()->route('login');
        }

        $userRole = auth()->user()->role->value;

        if (!in_array($userRole, $roles)) {
            abort(403, 'You do not have permission to access this page.');
        }

        return $next($request);
    }
}
```

Register the middleware in bootstrap/app.php :

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\RoleMiddleware::class,
    ]);
});
```

7. Routing Architecture

routes/web.php

```
<?php

use App\Http\Controllers\DashboardController;
use App\Http\Controllers\PatientController;
use Illuminate\Support\Facades\Route;

// — Public —
Route::get('/', function () {
    return redirect()->route('login');
});

// — Authenticated Routes —
Route::middleware(['auth', 'verified'])->group(function () {

    // Dashboard – role-based redirect
    Route::get('/dashboard', [DashboardController::class, 'index'])
        ->name('dashboard');

    // Patient routes – accessible by both roles
    Route::middleware('role:admin,doctor')->group(function () {
        Route::resource('patients', PatientController::class)
            ->except(['destroy']); // No deletion in Phase 0
    });
});

// Breeze auth routes (login, logout, etc.)
require __DIR__ . '/auth.php';
```

Route Summary

URL	Method	Controller	Access	Purpose
/	GET	—	Public	Redirects to login
/login	GET/ POST	Breeze	Public	Login form + auth
/dashboard	GET	DashboardController@index	Auth	Role-based dashboard
/patients	GET	PatientController@index	Admin, Doctor	Patient listing
/patients/ create	GET	PatientController@create	Admin, Doctor	Registration form

URL	Method	Controller	Access	Purpose
/patients	POST	PatientController@store	Admin, Doctor	Save new patient
/patients/{id}	GET	PatientController@show	Admin, Doctor	View patient detail
/patients/{id}/edit	GET	PatientController@edit	Admin, Doctor	Edit form
/patients/{id}	PUT	PatientController@update	Admin, Doctor	Save edits

8. Controllers, Services & Repositories

Your project already has the Repository + Service pattern scaffolded. Use it. The flow is:

```
HTTP Request → Controller → Service → Repository → Database
HTTP Response ← Controller ← Service ← Repository ← Database
```

PatientRepository

Handles all database queries for patients. Extends your existing BaseRepository.

app/Repositories/PatientRepository.php

```
<?php

namespace App\Repositories;

use App\Models\Patient;
use Illuminate\Contracts\Pagination\LengthAwarePaginator;

class PatientRepository extends BaseRepository
{
    public function __construct(Patient $model)
    {
        parent::__construct($model);
    }

    /**
     * Get paginated patients with optional search and filter.
     */
    public function getPaginated()
```

```

        ?string $search = null,
        ?string $bloodGroup = null,
        int $perPage = 15
    ): LengthAwarePaginator {
        $query = $this->model->with('registeredBy');

        if ($search) {
            $query->where(function ($q) use ($search) {
                $q->where('full_name', 'like', "%{$search}%")
                ->orWhere('phone', 'like', "%{$search}%")
                ->orWhere('patient_id', 'like', "%{$search}%");
            });
        }

        if ($bloodGroup) {
            $query->where('blood_group', $bloodGroup);
        }

        return $query->latest()->paginate($perPage);
    }

    /**
     * Get recent patients with limit.
     */
    public function getRecent(int $limit = 10)
    {
        return $this->model
            ->with('registeredBy')
            ->latest()
            ->take($limit)
            ->get();
    }

    /**
     * Count patients created today.
     */
    public function countToday(): int
    {
        return $this->model->whereDate('created_at', today())->count();
    }

    /**
     * Count patients created this week.
     */
    public function countThisWeek(): int
    {
        return $this->model->whereBetween('created_at', [
            now()->startOfWeek(),
            now()->endOfWeek(),
        ])->count();
    }

```

```

/**
 * Count patients with a given patient_id prefix (for ID generation).
 */
public function countByPatientIdPrefix(string $prefix): int
{
    return $this->model->where('patient_id', 'like', $prefix . '%')->count();
}
}

```

PatientService

Handles business logic — ID generation, creation, updates. Calls the repository for data access.

app/Services/PatientService.php

```

<?php

namespace App\Services;

use App\Models\Patient;
use App\Repositories\PatientRepository;
use Illuminate\Contracts\Pagination\LengthAwarePaginator;

class PatientService extends BaseService
{
    public function __construct(
        protected PatientRepository $patientRepository
    ) {}

    /**
     * Get paginated patient list with search/filter.
     */
    public function getPatientList(
        ?string $search = null,
        ?string $bloodGroup = null
    ): LengthAwarePaginator {
        return $this->patientRepository->getPaginated($search, $bloodGroup);
    }

    /**
     * Create a new patient with auto-generated ID.
     */
    public function createPatient(array $data, int $registeredBy): Patient
    {
        $data['patient_id'] = $this->generatePatientId();
        $data['registered_by'] = $registeredBy;

        return $this->patientRepository->create($data);
    }
}

```



```

/**
 * Update an existing patient.
 */
public function updatePatient(Patient $patient, array $data): Patient
{
    $patient->update($data);
    return $patient->fresh();
}

/**
 * Generate a unique patient ID: ACH-YYYYMMDD-XXXX
 */
private function generatePatientId(): string
{
    $today = now()->format('Ymd');
    $prefix = "ACH-{$today}-";

    $todayCount = $this->patientRepository->countByPatientIdPrefix($prefix);
    $sequence = str_pad($todayCount + 1, 4, '0', STR_PAD_LEFT);

    return $prefix . $sequence;
}
}

```

DashboardService

app/Services/DashboardService.php

```

<?php

namespace App\Services;

use App\Repositories\PatientRepository;

class DashboardService extends BaseService
{
    public function __construct(
        protected PatientRepository $patientRepository
    ) {}

    /**
     * Get admin dashboard statistics.
     */
    public function getAdminStats(): array
    {
        return [
            'total_patients' => $this->patientRepository->count(),
            'today_registered' => $this->patientRepository->countToday(),
            'this_week' => $this->patientRepository->countThisWeek(),
        ];
    }
}

```

```

    }

    /**
     * Get recent patients for dashboard display.
     */
    public function getRecentPatients(int $limit = 10)
    {
        return $this->patientRepository->getRecent($limit);
    }
}

```

DashboardController

Thin controller — delegates to DashboardService.

app/Http/Controllers/DashboardController.php

```

<?php

namespace App\Http\Controllers;

use App\Services\DashboardService;
use Illuminate\Http\Request;

class DashboardController extends BaseController
{
    public function __construct(
        protected DashboardService $dashboardService
    ) {}

    public function index(Request $request)
    {
        $user = $request->user();
        $recentPatients = $this->dashboardService->getRecentPatients();

        if ($user->isAdmin()) {
            $stats = $this->dashboardService->getAdminStats();
            return view('dashboard.admin', compact('stats', 'recentPatients'));
        }

        return view('dashboard.doctor', compact('recentPatients'));
    }
}

```

PatientController

Thin controller — delegates to PatientService.

app/Http/Controllers/PatientController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Patient;
use App\Services\PatientService;
use App\Http\Requests\StorePatientRequest;
use App\Http\Requests\UpdatePatientRequest;
use Illuminate\Http\Request;

class PatientController extends BaseController
{
    public function __construct(
        protected PatientService $patientService
    ) {}

    public function index(Request $request)
    {
        $patients = $this->patientService->getPatientList(
            search: $request->input('search'),
            bloodGroup: $request->input('blood_group')
        );

        return view('patients.index', compact('patients'));
    }

    public function create()
    {
        return view('patients.create');
    }

    public function store(StorePatientRequest $request)
    {
        $patient = $this->patientService->createPatient(
            data: $request->validated(),
            registeredBy: auth()->id()
        );

        return redirect()
            ->route('patients.show', $patient)
            ->with('success', 'Patient registered successfully. ID: ' . $patient->pa

    }

    public function show(Patient $patient)
    {
        $patient->load('registeredBy');
        return view('patients.show', compact('patient'));
    }
}
```

```

    public function edit(Patient $patient)
    {
        return view('patients.edit', compact('patient'));
    }

    public function update(UpdatePatientRequest $request, Patient $patient)
    {
        $this->patientService->updatePatient($patient, $request->validated());

        return redirect()
            ->route('patients.show', $patient)
            ->with('success', 'Patient record updated successfully.');
```

Patient Model

app/Models/Patient.php

```

<?php

namespace App\Models;

use App\Enums\BloodGroup;
use Illuminate\Database\Eloquent\Model;

class Patient extends Model
{
    protected $fillable = [
        'full_name',
        'date_of_birth',
        'phone',
        'blood_group',
        'allergies',
        'patient_id',
        'registered_by',
    ];

    protected function casts(): array
    {
        return [
            'date_of_birth' => 'date',
            'blood_group'   => BloodGroup::class,
        ];
    }

    // — Relationships —

    public function registeredBy()
    {
```

```
        return $this->belongsTo(User::class, 'registered_by');
    }

    // — Accessors —

    public function getAgeAttribute(): int
    {
        return $this->date_of_birth->age;
    }
}
```

Note: The Patient model does NOT have auto-generating ID logic in its `boot()` method. That logic lives in `PatientService::generatePatientId()` — keeping the model clean and the business logic in the service layer where it belongs.

9. Blade Views & UI Architecture

Master Layout Concept

Every page shares the same layout: a fixed sidebar on the left, a top bar at the top, and a content area.

TOPBAR: AfriChart logo "Welcome, Dr. [Name]" Logout	
SIDEBAR	CONTENT AREA
Dashboard Patients	(dashboard stats / patient list / registration form / etc.)

Layout Skeleton

`resources/views/layouts/app.blade.php`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>@yield('title', 'AfriChart EMR')</title>
@vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="bg-gray-50 font-sans antialiased">
  <div class="min-h-screen flex">

    {{-- Sidebar --}}
    <x-sidebar />

    {{-- Main Content --}}
    <div class="flex-1 flex flex-col">

      {{-- Topbar --}}
      <x-topbar />

      {{-- Flash Messages --}}
      @if(session('success'))
        <div class="mx-6 mt-4 bg-green-50 border border-green-200 text-green-600 px-4 py-3 rounded-lg text-sm">
          {{ session('success') }}
        </div>
      @endif

      {{-- Page Content --}}
      <main class="flex-1 p-6">
        @yield('content')
      </main>
    </div>
  </div>
</body>
</html>

```

Sidebar Component

resources/views/components/sidebar.blade.php

```

<aside class="w-64 bg-white border-r border-gray-200 min-h-screen flex flex-col">
  {{-- Logo --}}
  <div class="px-6 py-5 border-b border-gray-200">
    <div class="flex items-center gap-3">
      <div class="w-8 h-8 bg-blue-600 rounded-lg flex items-center justify-center text-white font-bold text-sm">A</div>
      <span class="text-lg font-semibold text-gray-900">AfriChart</span>
    </div>
  </div>

  {{-- Navigation --}}
  <nav class="flex-1 px-4 py-4 space-y-1">
    <a href="{{ route('dashboard') }}"

```

```

        class="flex items-center gap-3 px-3 py-2 rounded-lg text-sm font-medium
            {{ request()->routeIs('dashboard') ? 'bg-blue-50 text-blue-700' :
        <x-phosphor-house class="w-5 h-5" />
        Dashboard
    </a>

    <a href="{{ route('patients.index') }}"
        class="flex items-center gap-3 px-3 py-2 rounded-lg text-sm font-medium
            {{ request()->routeIs('patients.*') ? 'bg-blue-50 text-blue-700' :
        <x-phosphor-users class="w-5 h-5" />
        Patients
    </a>
</nav>

{{-- User Info --}}
<div class="px-4 py-4 border-t border-gray-200">
    <div class="flex items-center gap-3 px-3">
        <div class="w-8 h-8 bg-gray-200 rounded-full flex items-center
            justify-center text-gray-600 text-xs font-medium">
            {{ strtoupper(substr(auth()->user()->name, 0, 2)) }}
        </div>
        <div class="flex-1 min-w-0">
            <p class="text-sm font-medium text-gray-900 truncate">
                {{ auth()->user()->name }}
            </p>
            <p class="text-xs text-gray-500">
                {{ auth()->user()->role->label() }}
            </p>
        </div>
    </div>
</div>
</aside>

```

Topbar Component

resources/views/components/topbar.blade.php

```

<header class="bg-white border-b border-gray-200 px-6 py-3 flex items-center justify
    <div>
        <h1 class="text-lg font-semibold text-gray-900">
            @yield('page-title', 'Dashboard')
        </h1>
        <p class="text-sm text-gray-500">
            @yield('page-subtitle', '')
        </p>
    </div>

    <div class="flex items-center gap-4">
        <span class="text-sm text-gray-600">
            {{ now()->format('l, j F Y') }}
        </span>
    </div>

```

```

</span>

<form method="POST" action="{{ route('logout') }}">
    @csrf
    <button type="submit" class="text-sm text-gray-500 hover:text-gray-700
        flex items-center gap-1">
        <x-phosphor-sign-out class="w-4 h-4" />
        Logout
    </button>
</form>
</div>
</header>

```

Admin Dashboard

resources/views/dashboard/admin.blade.php

```

@extends('layouts.app')
@section('title', 'Admin Dashboard – AfriChart EMR')
@section('page-title', 'Admin Dashboard')
@section('page-subtitle', 'Overview of clinic activity')

@section('content')
    {{-- Stat Cards Row --}}
    <div class="grid grid-cols-3 gap-6 mb-8">
        <x-stat-card
            title="Total Patients"
            :value="$stats['total_patients']"
            icon="phosphor-users" />
        <x-stat-card
            title="Registered Today"
            :value="$stats['today_registered']"
            icon="phosphor-user-plus" />
        <x-stat-card
            title="This Week"
            :value="$stats['this_week']"
            icon="phosphor-calendar" />
    </div>

    {{-- Recent Patients --}}
    <div class="bg-white rounded-xl border border-gray-200">
        <div class="px-6 py-4 border-b border-gray-200 flex items-center justify-between">
            <h2 class="text-base font-semibold text-gray-900">Recent Registrations</h2>
            <a href="{{ route('patients.create') }}"
                class="text-sm text-blue-600 hover:text-blue-700 font-medium">
                + Register Patient
            </a>
        </div>
        <x-patient-table :patients="$recentPatients" />
    </div>

```



```
@endsection
```

Patient Registration Form

resources/views/patients/create.blade.php

```
@extends('layouts.app')
@section('title', 'Register Patient – AfriChart EMR')
@section('page-title', 'Register New Patient')

@section('content')
    <div class="max-w-2xl">
        <div class="bg-white rounded-xl border border-gray-200 p-6">
            <form method="POST" action="{{ route('patients.store') }}">
                @csrf

                {{-- Full Name --}}
                <div class="mb-5">
                    <label for="full_name" class="block text-sm font-medium text-gray-700">
                        Full Name
                    </label>
                    <input type="text" name="full_name" id="full_name"
                        value="{{ old('full_name') }}"
                        class="w-full rounded-lg border-gray-300 shadow-sm
                            focus:border-blue-500 focus:ring-blue-500 text-sm"
                        required />
                    @error('full_name')
                        <p class="mt-1 text-sm text-red-600">{{ $message }}</p>
                    @enderror
                </div>

                {{-- Date of Birth --}}
                <div class="mb-5">
                    <label for="date_of_birth" class="block text-sm font-medium text-gray-700">
                        Date of Birth
                    </label>
                    <input type="date" name="date_of_birth" id="date_of_birth"
                        value="{{ old('date_of_birth') }}"
                        max="{{ now()->format('Y-m-d') }}"
                        class="w-full rounded-lg border-gray-300 shadow-sm
                            focus:border-blue-500 focus:ring-blue-500 text-sm"
                        required />
                    @error('date_of_birth')
                        <p class="mt-1 text-sm text-red-600">{{ $message }}</p>
                    @enderror
                </div>

                {{-- Phone Number --}}
                <div class="mb-5">
                    <label for="phone" class="block text-sm font-medium text-gray-700">
```

```

        Phone Number
    </label>
    <input type="tel" name="phone" id="phone"
        value="{{ old('phone') }}"
        placeholder="08012345678"
        class="w-full rounded-lg border-gray-300 shadow-sm
            focus:border-blue-500 focus:ring-blue-500 text-sm"
        required />
    @error('phone')
        <p class="mt-1 text-sm text-red-600">{{ $message }}</p>
    @enderror
</div>

{{-- Blood Group --}}
<div class="mb-5">
    <label for="blood_group" class="block text-sm font-medium text-gray-900">
        Blood Group
    </label>
    <select name="blood_group" id="blood_group"
        class="w-full rounded-lg border-gray-300 shadow-sm
            focus:border-blue-500 focus:ring-blue-500 text-sm"
        required>
        <option value="">Select blood group</option>
        @foreach(\App\Enums\BloodGroup::cases() as $group)
            <option value="{{ $group->value }}"
                {{ old('blood_group') === $group->value ? 'selected' : '' }}
                {{ $group->label() }}
            </option>
        @endforeach
    </select>
    @error('blood_group')
        <p class="mt-1 text-sm text-red-600">{{ $message }}</p>
    @enderror
</div>

{{-- Allergies --}}
<div class="mb-6">
    <label for="allergies" class="block text-sm font-medium text-gray-900">
        Known Allergies
        <span class="text-gray-400 font-normal">(optional)</span>
    </label>
    <textarea name="allergies" id="allergies" rows="3"
        placeholder="e.g. Penicillin, dust, latex..."
        class="w-full rounded-lg border-gray-300 shadow-sm
            focus:border-blue-500 focus:ring-blue-500 text-sm"
        >{{ old('allergies') }}</textarea>
    @error('allergies')
        <p class="mt-1 text-sm text-red-600">{{ $message }}</p>
    @enderror
</div>

{{-- Submit --}}

```

```

    <div class="flex items-center gap-3">
      <button type="submit"
        class="bg-blue-600 text-white px-5 py-2.5 rounded-lg
          text-sm font-medium hover:bg-blue-700
          focus:outline-none focus:ring-2 focus:ring-blue-5
          focus:ring-offset-2 transition-colors">
        Register Patient
      </button>
      <a href="{ route('patients.index') }"
        class="text-sm text-gray-500 hover:text-gray-700">
        Cancel
      </a>
    </div>
  </form>
</div>
</div>
@endsection
```

10. Design Direction

Visual Identity

Since there are no mockups from the client, you own the design. The system needs to feel clinical, trustworthy, and easy to use for non-technical staff.

Element	Value
Primary colour	#2563EB (Tailwind blue-600) — trustworthy, clinical, professional
Background	#F9FAFB (gray-50) — soft off-white, reduces eye strain
Card background	FFFFFF with border-gray-200
Text primary	#111827 (gray-900)
Text secondary	#6B7280 (gray-500)
Font	System font stack (Tailwind default) — loads instantly, universally readable
Border radius	rounded-lg (8px) — modern but not playful
Spacing	Consistent p-6 for card padding, gap-6 for grid gaps

UI Principles for This Project

1. **No clutter** — Every element on screen should serve a purpose. If a receptionist would be confused by it, remove it.
2. **Big click targets** — Buttons should be easy to click. Use `px-5 py-2.5` minimum.
3. **Immediate feedback** — Flash messages on every action (patient created, updated).
4. **Readable tables** — Alternating row colours (`even:bg-gray-50`), adequate cell padding.
5. **Labels above inputs** — Not floating labels, not placeholders-as-labels. Clear, visible labels.
6. **Error states** — Red text below the offending field, not a blob at the top of the form.

What NOT to Do

- No dark mode. Desktop-optimised, light theme only.
- No animations beyond hover states.
- No gradients, no shadows heavier than `shadow-sm`.
- No sidebar collapse/hamburger menu. It's desktop — the sidebar is always visible.
- No custom fonts. System defaults are faster and more reliable.

11. Validation Rules

StorePatientRequest

Extends your existing BaseRequest.

app/Http/Requests/StorePatientRequest.php

```
<?php

namespace App\Http\Requests;

use App\Enums\BloodGroup;
use Illuminate\Validation\Rules\Enum;

class StorePatientRequest extends BaseRequest
{
    public function authorize(): bool
    {
        return true; // Role check is handled by middleware
    }

    public function rules(): array
    {
        return [
            'full_name' => ['required', 'string', 'max:255', 'min:3'],
            'date_of_birth' => ['required', 'date', 'before_or_equal:today'],
        ];
    }
}
```

```

        'phone'            => [
            'required',
            'string',
            'max:20',
            'regex:/^(\+234|0)[789]\d{9}$/', // Nigerian phone format
        ],
        'blood_group'      => ['required', new Enum(BloodGroup::class)],
        'allergies'        => ['nullable', 'string', 'max:1000'],
    ];
}

public function messages(): array
{
    return [
        'phone.regex'            => 'Enter a valid Nigerian phone number',
        'date_of_birth.before_or_equal' => 'Date of birth cannot be in the future',
        'full_name.min'          => 'Please enter the patient\'s full name',
    ];
}
}

```

UpdatePatientRequest

app/Http/Requests/UpdatePatientRequest.php

Same rules as Store. Separate class so you can diverge later in Phase 1.

```

<?php

namespace App\Http\Requests;

use App\Enums\BloodGroup;
use Illuminate\Validation\Rules\Enum;

class UpdatePatientRequest extends BaseRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'full_name'      => ['required', 'string', 'max:255', 'min:3'],
            'date_of_birth' => ['required', 'date', 'before_or_equal:today'],
            'phone'          => [
                'required',
                'string',
                'max:20',
            ],
        ];
    }
}

```

```

        'regex:/^(\+234|0)[789]\d{9}$/',
    ],
    'blood_group'    => ['required', new Enum(BloodGroup::class)],
    'allergies'      => ['nullable', 'string', 'max:1000'],
    ];
}

public function messages(): array
{
    return [
        'phone.regex'          => 'Enter a valid Nigerian phone number',
        'date_of_birth.before_or_equal' => 'Date of birth cannot be in the futur
    ];
}
}

```

12. Seeders & Demo Data

The client will test this with demo data. Make it realistic — not "John Doe" and "Test Patient."

UserSeeder

database/seeders/UserSeeder.php

```

<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name'      => 'Admin User',
            'email'     => 'admin@africhart.com',
            'password'  => bcrypt('password'),
            'role'      => 'admin',
        ]);

        User::create([
            'name'      => 'Dr. Emeka Okafor',
            'email'     => 'doctor@africhart.com',
            'password'  => bcrypt('password'),
            'role'      => 'doctor',
        ]);
    }
}

```

```

    });
}
}

```

PatientSeeder

database/seeder/PatientSeeder.php

Use realistic Nigerian names and data. Seed 25-30 patients.

```

<?php

namespace Database\Seeders;

use App\Models\Patient;
use App\Models\User;
use App\Services\PatientService;
use Illuminate\Database\Seeder;

class PatientSeeder extends Seeder
{
    public function run(): void
    {
        $admin = User::where('role', 'admin')->first();
        $patientService = app(PatientService::class);

        $patients = [
            ['full_name' => 'Chioma Adaeze Nwosu', 'date_of_birth' => '1990-03-15'],
            ['full_name' => 'Emeka Chukwuemeka Obi', 'date_of_birth' => '1985-07-22'],
            ['full_name' => 'Fatima Bello Mohammed', 'date_of_birth' => '1978-11-08'],
            ['full_name' => 'Oluwaseun David Adeyemi', 'date_of_birth' => '1995-05-10'],
            ['full_name' => 'Amina Yusuf Ibrahim', 'date_of_birth' => '2000-02-28'],
            ['full_name' => 'Chinedu Ikenna Eze', 'date_of_birth' => '1982-09-03'],
            ['full_name' => 'Aisha Bintu Abubakar', 'date_of_birth' => '1993-12-01'],
            ['full_name' => 'Tunde Babatunde Ogunleye', 'date_of_birth' => '1975-06-18'],
            ['full_name' => 'Ngozi Blessing Okonkwo', 'date_of_birth' => '1988-04-05'],
            ['full_name' => 'Yakubu Musa Danladi', 'date_of_birth' => '1970-08-20'],
            ['full_name' => 'Funke Adebisi Oladipo', 'date_of_birth' => '1997-01-12'],
            ['full_name' => 'Obinna Francis Nwankwo', 'date_of_birth' => '1991-10-07'],
            ['full_name' => 'Hauwa Sadiya Garba', 'date_of_birth' => '2003-03-25'],
            ['full_name' => 'Adewale Samuel Akinola', 'date_of_birth' => '1986-07-14'],
            ['full_name' => 'Uchenna Grace Okoro', 'date_of_birth' => '1999-02-09'],
            ['full_name' => 'Ibrahim Suleiman Bako', 'date_of_birth' => '1968-11-30'],
            ['full_name' => 'Yetunde Folake Bakare', 'date_of_birth' => '1994-05-21'],
            ['full_name' => 'Kelechi Promise Okafor', 'date_of_birth' => '2001-09-04'],
            ['full_name' => 'Maryam Zainab Abdullahi', 'date_of_birth' => '1980-12-17'],
            ['full_name' => 'Segun Olalekan Adekunle', 'date_of_birth' => '1973-04-26'],
            ['full_name' => 'Ifeoma Patricia Uche', 'date_of_birth' => '1996-08-11'],
            ['full_name' => 'Abdulrahman Idris Musa', 'date_of_birth' => '1987-06-03'],
            ['full_name' => 'Blessing Chidinma Obi', 'date_of_birth' => '2005-01-19'],

```

```

        ['full_name' => 'Sani Kabiru Yusuf',           'date_of_birth' => '1965-1
        ['full_name' => 'Omolara Deborah Ajayi',       'date_of_birth' => '1992-0
    ];

    foreach ($patients as $data) {
        $patientService->createPatient($data, $admin->id);
    }
}
}

```

DatabaseSeeder

database/seeder/DatabaseSeeder.php

```

<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            UserSeeder::class,
            PatientSeeder::class,
        ]);
    }
}

```

Run with:

```
php artisan migrate:fresh --seed
```

13. Search & Filter Implementation

The patient listing page needs text search and blood group filter. Both work via query parameters so the URL is shareable.

URL: `/patients?search=chioma&blood_group=0%2B`

The `PatientRepository::getPaginated()` method (Section 8) handles the query logic. The view needs a search form above the table:


```

{{-- Search & Filter Bar --}}
<div class="bg-white rounded-xl border border-gray-200 p-4 mb-6">
  <form method="GET" action="{{ route('patients.index') }}"
    class="flex items-center gap-4">

    <div class="flex-1">
      <input type="text" name="search" value="{{ request('search') }}"
        placeholder="Search by name, phone, or patient ID..."
        class="w-full rounded-lg border-gray-300 shadow-sm
          focus:border-blue-500 focus:ring-blue-500 text-sm" />
    </div>

    <div class="w-40">
      <select name="blood_group"
        class="w-full rounded-lg border-gray-300 shadow-sm
          focus:border-blue-500 focus:ring-blue-500 text-sm">
        <option value="">All Blood Groups</option>
        @foreach(\App\Enums\BloodGroup::cases() as $group)
          <option value="{{ $group->value }}"
            {{ request('blood_group') === $group->value ? 'selected'
              {{ $group->label() }}
          </option>
        @endforeach
      </select>
    </div>

    <button type="submit"
      class="bg-blue-600 text-white px-4 py-2.5 rounded-lg text-sm
        font-medium hover:bg-blue-700 transition-colors">
      Search
    </button>

    @if(request('search') || request('blood_group'))
      <a href="{{ route('patients.index') }}"
        class="text-sm text-gray-500 hover:text-gray-700">
        Clear
      </a>
    @endif
  </form>
</div>

```

14. Security Considerations

Even for Phase 0, follow security fundamentals. The client is building an EMR — security is not optional.

Area	Implementation
CSRF	Laravel handles this automatically with <code>@csrf</code> in forms
SQL injection	Eloquent ORM prevents this by default. Never use raw queries.
XSS	Blade's <code>{{ }}</code> double-curly syntax escapes output automatically
Mass assignment	<code>\$fillable</code> on every model. Never use <code>\$guarded = []</code>
Password hashing	Laravel hashes passwords automatically via the <code>hashed</code> cast
Role enforcement	Middleware-level, not view-level. A user can't bypass it by visiting the URL directly.
Input validation	FormRequest classes validate before the controller ever sees the data
Sensitive data in logs	Don't log patient names or phone numbers. Keep logs generic.

Things to Explicitly Avoid

- Don't store passwords in plain text anywhere (including seeders in production — demo only)
- Don't expose user IDs in the URL for patients (use the patient's own `id`, not the `registered_by` user)
- Don't disable CSRF for any route
- Don't use `{!! !!}` (unescaped output) unless you're rendering trusted HTML

15. Testing Checklist

Before delivery, manually test every flow. Check off each item.

Authentication

- ☐ Admin can log in with `admin@africhart.com` / `password`
- ☐ Doctor can log in with `doctor@africhart.com` / `password`
- ☐ Wrong password shows error message
- ☐ Logged-out user is redirected to `/login`
- ☐ Logout works and redirects to login

Role-Based Access

- ☐ Admin sees admin dashboard after login

- ☐ Doctor sees doctor dashboard after login
- ☐ Both roles can access patient pages
- ☐ Manually visiting `/dashboard` shows correct view per role

Patient Registration

- ☐ Form displays all required fields
- ☐ Submitting valid data creates patient and redirects to show page
- ☐ Success message shows patient ID (e.g. ACH-20260530-0001)
- ☐ Submitting empty form shows validation errors
- ☐ Invalid phone number is rejected
- ☐ Future date of birth is rejected
- ☐ Blood group dropdown shows all 8 options

Patient Listing

- ☐ All seeded patients display in a table
- ☐ Pagination works (15 per page)
- ☐ Search by name returns correct results
- ☐ Search by phone returns correct results
- ☐ Search by patient ID returns correct results
- ☐ Blood group filter works
- ☐ Search + filter combined works
- ☐ Clear button resets search

Patient Edit

- ☐ Edit form is pre-filled with existing data
- ☐ Updating a field saves correctly
- ☐ Validation still runs on edit

UI / Design

- ☐ Login page shows AfriChart branding
- ☐ Sidebar highlights current page
- ☐ Sidebar shows user name and role
- ☐ Tables are readable with adequate spacing
- ☐ Flash messages appear and are styled correctly
- ☐ No broken layouts on a standard 1366x768 or 1920x1080 screen

Code Quality

- [] No `dd()` or `dump()` left in code
- [] No commented-out code blocks
- [] No `TODO` comments without resolution
- [] Every model has proper `$fillable`
- [] `.env` is in `.gitignore`
- [] `README.md` has setup instructions and demo credentials

16. Deployment & Demo Environment

The deliverable includes a working demo environment the client can access.

Option A: Ngrok (Quick but Temporary)

```
php artisan serve
ngrok http 8000
```

Problem: URL expires. Use this only for a live walkthrough call, not as a persistent demo.

Option B: Deploy to a VPS (Recommended)

If you have a VPS with your standard LAMP/LEMP setup:

```
# On your server
cd /var/www
git clone [your-repo-url] africhart-emr
cd africhart-emr

# Install dependencies
composer install --no-dev --optimize-autoloader
npm install && npm run build

# Environment setup
cp .env.example .env
php artisan key:generate
# Configure database in .env

# Migrate and seed
php artisan migrate --seed

# Set permissions
sudo chown -R www-data:www-data storage bootstrap/cache
```

```
sudo chmod -R 775 storage bootstrap/cache
```

```
# Configure Apache/Nginx vhost pointing to /var/www/africhart-emr/public
```

Set up a subdomain like `africhart.mgbah.dev` or similar. Clean, persistent, professional.

README.md Template

Include this in your repo root:

```
# AfriChart EMR — Phase 0
```

Electronic Medical Records system for private clinics in Nigeria.

Tech Stack

- Laravel 11 (PHP 8.3)
- Blade + Tailwind CSS
- MySQL 8.0

Architecture

Repository + Service layer pattern:

- Controllers handle HTTP
- Services handle business logic
- Repositories handle data access

Setup

1. Clone the repository
2. Run ``composer install``
3. Run ``npm install && npm run build``
4. Copy ``.env.example`` to ``.env`` and configure your database
5. Run ``php artisan key:generate``
6. Run ``php artisan migrate --seed``
7. Run ``php artisan serve``

Demo Credentials

Role	Email	Password
Admin	admin@africhart.com	password
Doctor	doctor@africhart.com	password

Features (Phase 0)

- Login with role-based access (Admin / Doctor)
- Separate dashboards per role
- Patient registration with validation
- Patient listing with search and blood group filter
- Patient profile editing

17. Day-by-Day Build Plan

Day	Focus	Deliverables
1	Foundation	Install Breeze + Tailwind + Phosphor Icons, switch to MySQL, new migrations (role column + patients table), Enums, Patient model, modify User model, Role middleware, Git commit
2	Service Layer	PatientRepository, PatientService, DashboardService — all business logic wired up and testable via tinker
3	Controllers & Routes	DashboardController, PatientController, all routes in web.php, form request validation classes
4	Layout & UI Shell	Master layout (app.blade.php), sidebar component, topbar component, stat-card component — the visual framework everything sits in
5	Patient Views	Registration form, patient listing page with table, patient detail view, edit form, search/filter bar
6	Dashboards & Seeders	Admin dashboard view with stats, Doctor dashboard view, UserSeeder with demo accounts, PatientSeeder with 25 realistic records, branded login page
7	Polish & Deploy	Testing (full checklist), bug fixes, UI polish, deploy demo environment, write README, final commit, share repo link with client

Commit Strategy

Commit after every meaningful unit of work:

```
feat: initial laravel 11 setup with breeze and tailwind
feat: add role column to users, create patients migration
feat: user role enum, blood group enum
feat: user model with role casting and helpers
feat: role middleware and route protection
feat: patient model and repository
feat: patient service with ID generation
feat: dashboard service with stats
feat: patient controller with full CRUD
feat: form request validation for patients
feat: dashboard controller with role routing
feat: master layout with sidebar and topbar
feat: africhart-branded login page
```

```
feat: admin and doctor dashboard views
feat: patient registration form
feat: patient listing with search and filter
feat: patient detail and edit views
feat: database seeders with demo data
fix: phone validation regex for nigerian numbers
chore: deploy configuration and README
```

18. Deliverables Checklist

Before you send the "Phase 0 is ready" message, every item must be checked:

- ☐ GitHub repo is accessible to the client
- ☐ README with setup instructions and demo credentials
- ☐ Working demo URL (persistent, not ngrok)
- ☐ Login system functional (both accounts)
- ☐ Admin dashboard shows patient stats
- ☐ Doctor dashboard shows recent patients
- ☐ Patient registration form with full validation
- ☐ Patient listing with search by name, phone, and patient ID
- ☐ Patient listing with blood group filter
- ☐ Patient edit functionality
- ☐ AfriChart branding on login and sidebar
- ☐ Clean, well-structured codebase (repository + service pattern)
- ☐ No debug code, no leftover comments
- ☐ Seeded with 25 realistic patient records

End of document. Build clean. Ship on time.